

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
data={
    "Student_id":["001","002","003","004","nan","006","007","008","009","010"],
    "Student_name":["Rahul","Rohan","Sonia","nan","Khushi","Komal","Dev","Devans",
    "Height_in_foot":[5.7,6,5.7,5.6,6,5.4,5.8,5,5.3,5],
    "Weight_in_kg":[60,70,55,67,66,57,70,62,59,50],
    "Semester":[1,2,3,2,4,1,2,4,5,3],
    "Gender":["M","M","F","M","F","F","M","F","F","F"],
    "City":["Meerut","Kotdwar","Rishikesh","Dehradun","Dehradun","Noida","nan","",
    "age":[18,20,19,23,22,19,17,22,20,21]
}
df= pd.DataFrame(data)
print(df)
```

	Student_id	Student_name	Height_in_foot	Weight_in_kg	Semester	Gender	\
0	001	Rahul	5.7	60	1	M	
1	002	Rohan	6.0	70	2	M	
2	003	Sonia	5.7	55	3	F	
3	004	nan	5.6	67	2	M	
4	nan	Khushi	6.0	66	4	F	
5	006	Komal	5.4	57	1	F	
6	007	Dev	5.8	70	2	M	
7	008	Devanshi	5.0	62	4	F	
8	009	Bharti	5.3	59	5	F	
9	010	Tia	5.0	50	3	F	

	City	age
0	Meerut	18
1	Kotdwar	20
2	Rishikesh	19
3	Dehradun	23
4	Dehradun	22
5	Noida	19
6	nan	17
7	Kotdwar	22
8	Saharanpur	20
9	saharanpur	21

```
In [3]: print(df.shape)
```

```
(10, 8)
```

```
In [4]: list(df)
```

```
Out[4]: ['Student_id',
'Student_name',
'Height_in_foot',
'Weight_in_kg',
'Semester',
'Gender',
'City',
'age']
```

```
In [5]: df.isnull()
```

Out[5]:

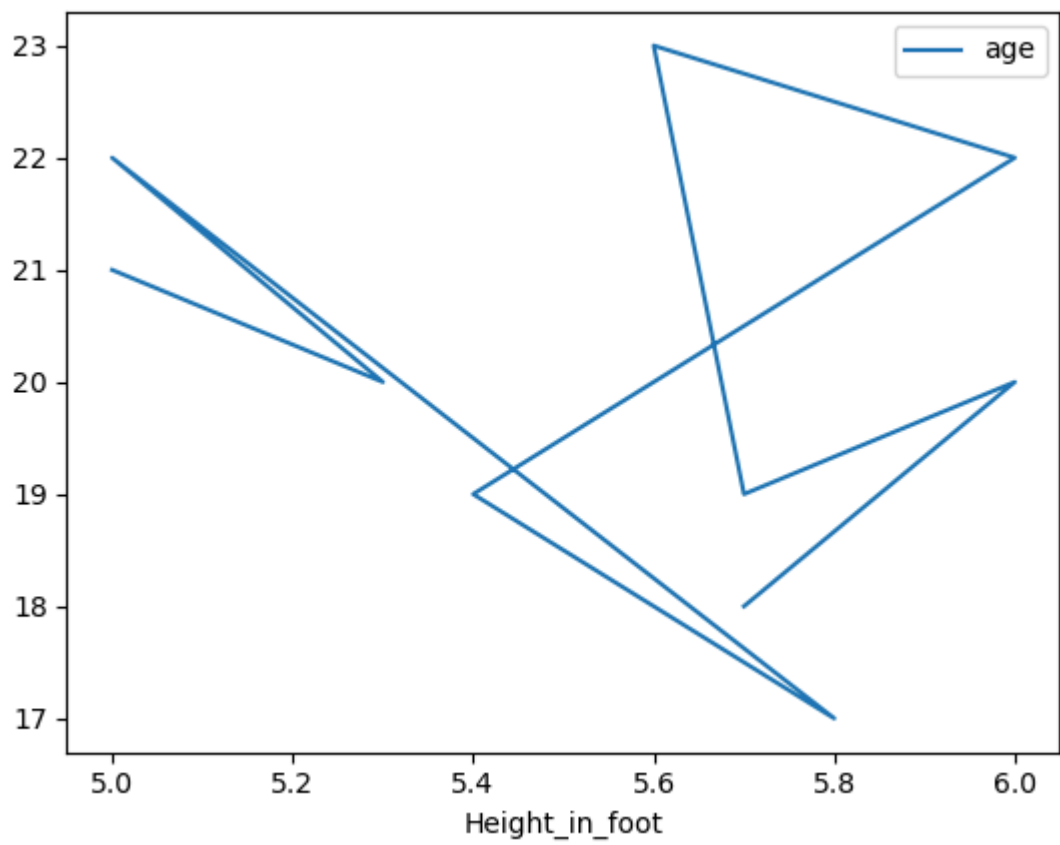
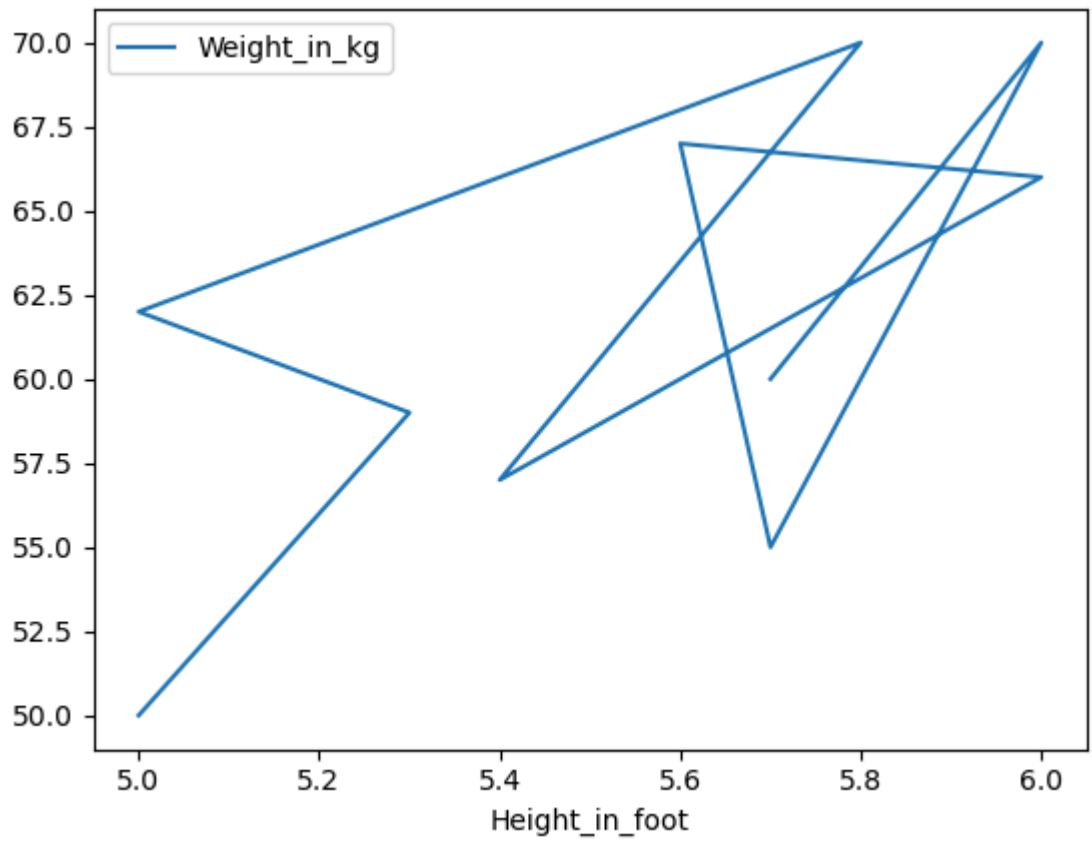
	Student_id	Student_name	Height_in_foot	Weight_in_kg	Semester	Gender	City
0	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False
5	False	False	False	False	False	False	False
6	False	False	False	False	False	False	False
7	False	False	False	False	False	False	False
8	False	False	False	False	False	False	False
9	False	False	False	False	False	False	False

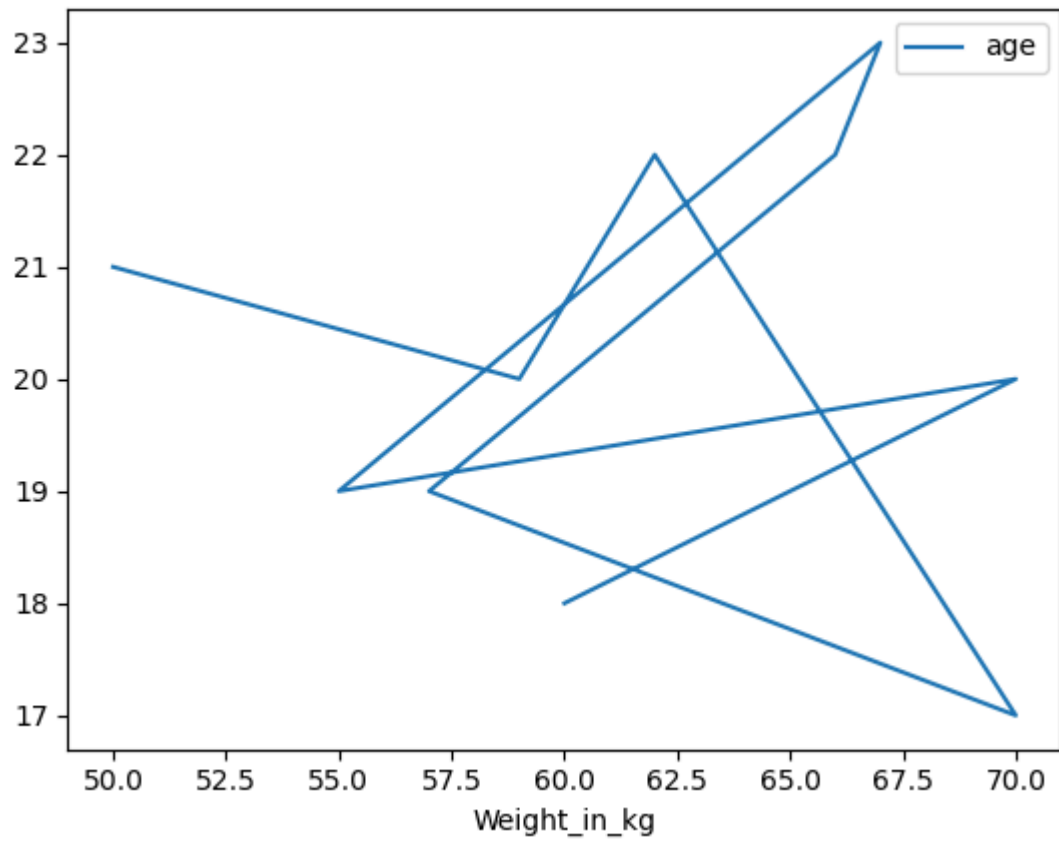
In [6]: `#unique_names`
`print("\nUnique student names:",df["Student_name"].unique())`

Unique student names: <bound method Series.unique of 0 Rahul
 1 Rohan
 2 Sonia
 3 nan
 4 Khushi
 5 Komal
 6 Dev
 7 Devanshi
 8 Bharti
 9 Tia
 Name: Student_name, dtype: object>

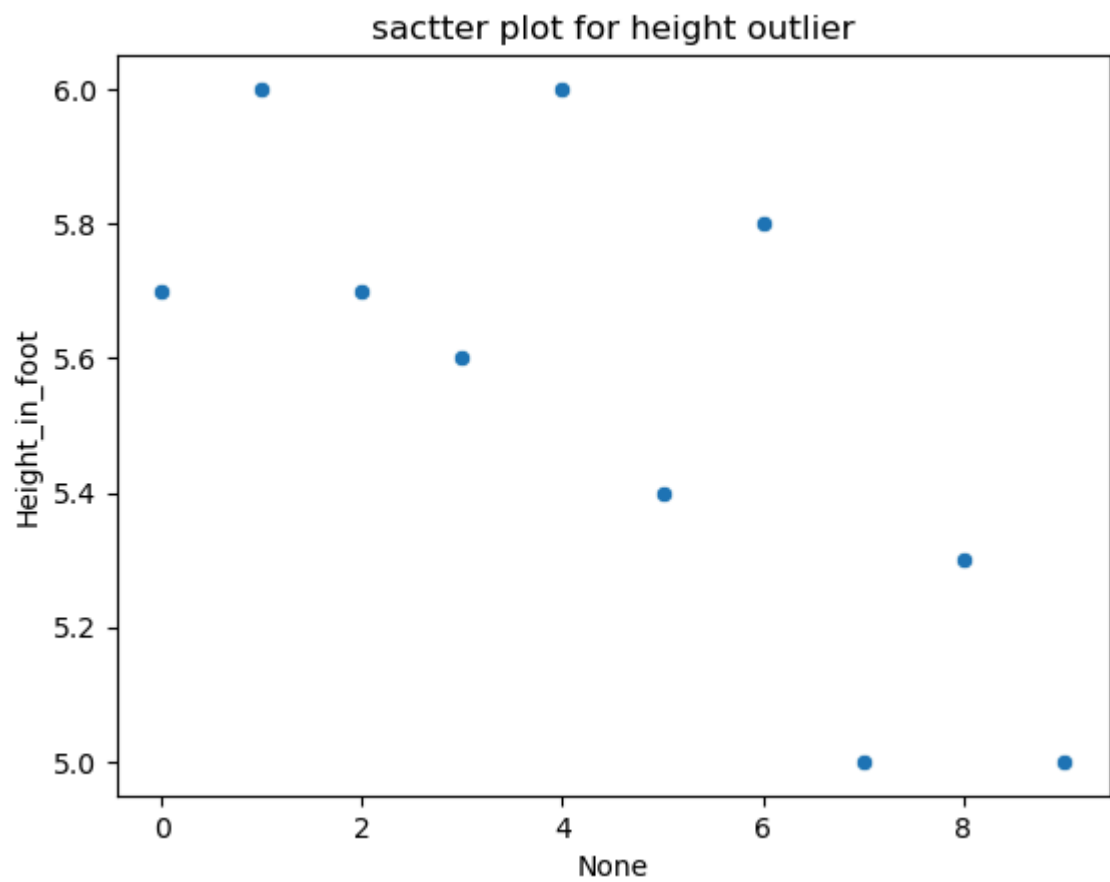
In [7]: `df.plot(x="Height_in_foot",y="Weight_in_kg",kind="line")`
`df.plot(x="Height_in_foot",y="age",kind="line")`
`df.plot(x="Weight_in_kg",y="age",kind="line")`

Out[7]: <Axes: xlabel='Weight_in_kg'>

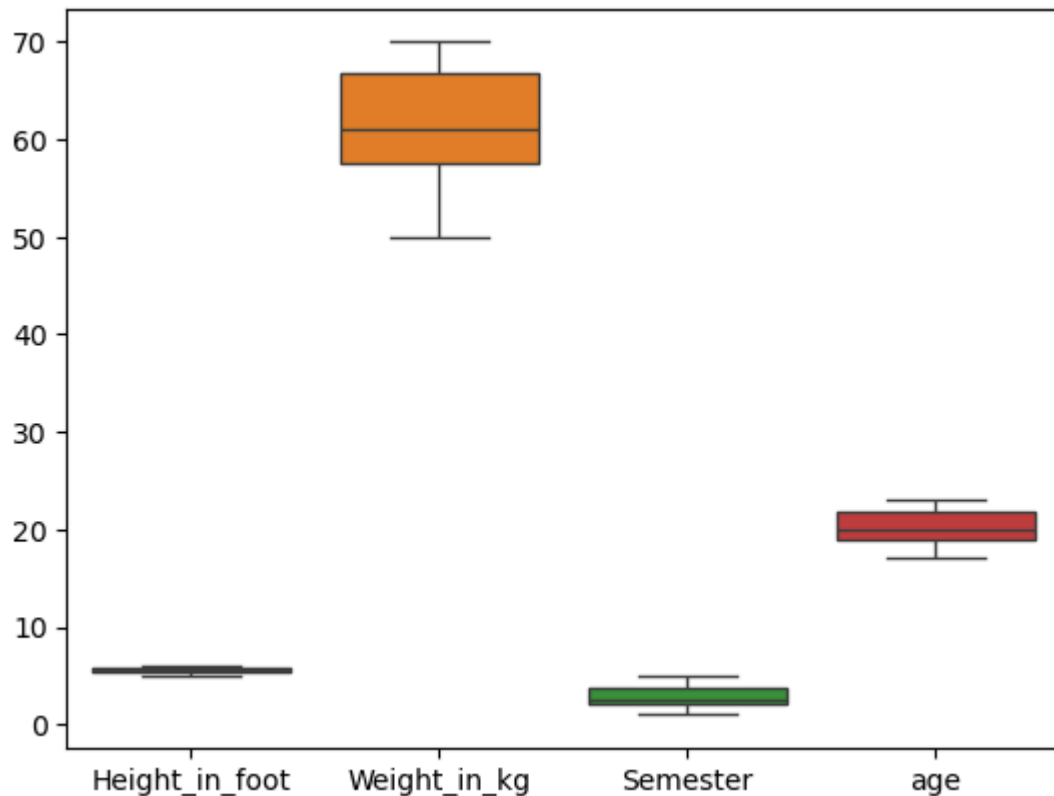




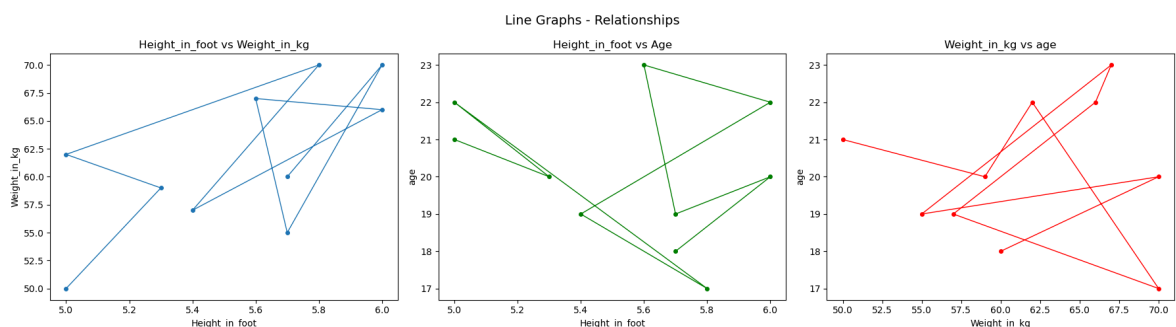
```
In [8]: sns.scatterplot(x=df.index,y='Height_in_foot',data=df)
plt.title("sactter plot for height outlier")
plt.show()
```



```
In [9]: sns.boxplot(data=df)
plt.show()
```



```
In [12]: fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle('Line Graphs - Relationships', fontsize=14)
axes[0].plot(df['Height_in_foot'], df['Weight_in_kg'], 'o-', linewidth=1, marker
axes[0].set_title('Height_in_foot vs Weight_in_kg')
axes[0].set_xlabel('Height_in_foot')
axes[0].set_ylabel('Weight_in_kg')
axes[1].plot(df['Height_in_foot'], df['age'], 'o-', linewidth=1, markersize=4, c
axes[1].set_title('Height_in_foot vs Age')
axes[1].set_xlabel('Height_in_foot')
axes[1].set_ylabel('age')
axes[2].plot(df['Weight_in_kg'], df['age'], 'o-', linewidth=1, markersize=4, col
axes[2].set_title('Weight_in_kg vs age')
axes[2].set_xlabel('Weight_in_kg')
axes[2].set_ylabel('age')
plt.tight_layout()
plt.show()
```



```
In [ ]:
```

```
In [ ]:
```

In []:

```

In [20]: # Question 02
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
plt.style.use('seaborn-v0_8')
sns.set_palette("pastel")
# i. Create dataset and compute BMI
np.random.seed(42)
n = 100
data = {
    'height': np.random.normal(165, 8, n), # cm
    'weight': np.random.normal(65, 12, n), # kg
    'gender': np.random.choice(['Male', 'Female'], n)
}
null_indices = np.random.choice(n, 10, replace=False)
data['gender'][null_indices] = np.nan
df = pd.DataFrame(data)
# Compute BMI: BMI = weight(kg) / height(m)^2
df['BMI'] = df['weight'] / ((df['height']/100)**2)

print("BMI computed using DataFrame operations")
print("Sample data:")
print(df.head())
print()

```

BMI computed using DataFrame operations

Sample data:

	height	weight	gender	BMI
0	168.973713	48.015551	Female	16.816810
1	163.893886	59.952256	Female	22.319272
2	170.181508	60.887426	Male	21.023396
3	177.184239	55.372673	Female	17.637836
4	163.126773	63.064571	Male	23.699269

```

In [21]: filename = 'bmi_data.csv'
df.to_csv(filename, index=False)
print(f"Dataset saved as '{filename}'")

```

Dataset saved as 'bmi_data.csv'

```

In [22]: df_loaded = pd.read_csv(filename)
print("CSV imported successfully")
print("Loaded data shape:", df_loaded.shape)

```

CSV imported successfully

Loaded data shape: (100, 4)

```

In [23]: print("DataFrame Info:")
print(f"Shape: {df_loaded.shape}")
print(f"Size: {df_loaded.size}")

```

DataFrame Info:

Shape: (100, 4)

Size: 400

```

In [19]: null_count = df_loaded['gender'].isnull().sum()

```

```
print(f"Null values in Gender column:{null_count}")
```

Null values in Gender column:10

```
In [24]: df_clean = df_loaded.dropna(subset=['gender'])
print(f"After removing nulls: {len(df_clean)} rows (removed {len(df_loaded)-len(df_clean)})")
```

After removing nulls: 90 rows (removed 10)

```
In [25]: print("Dataset Information:")
print("Structure:")
print(df_clean.info())
print("Statistical Summary:")
print(df_clean.describe())
```

Dataset Information:

Structure:

```
<class 'pandas.core.frame.DataFrame'>
```

Index: 90 entries, 0 to 99

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	height	90 non-null	float64
1	weight	90 non-null	float64
2	gender	90 non-null	object
3	BMI	90 non-null	float64

dtypes: float64(3), object(1)

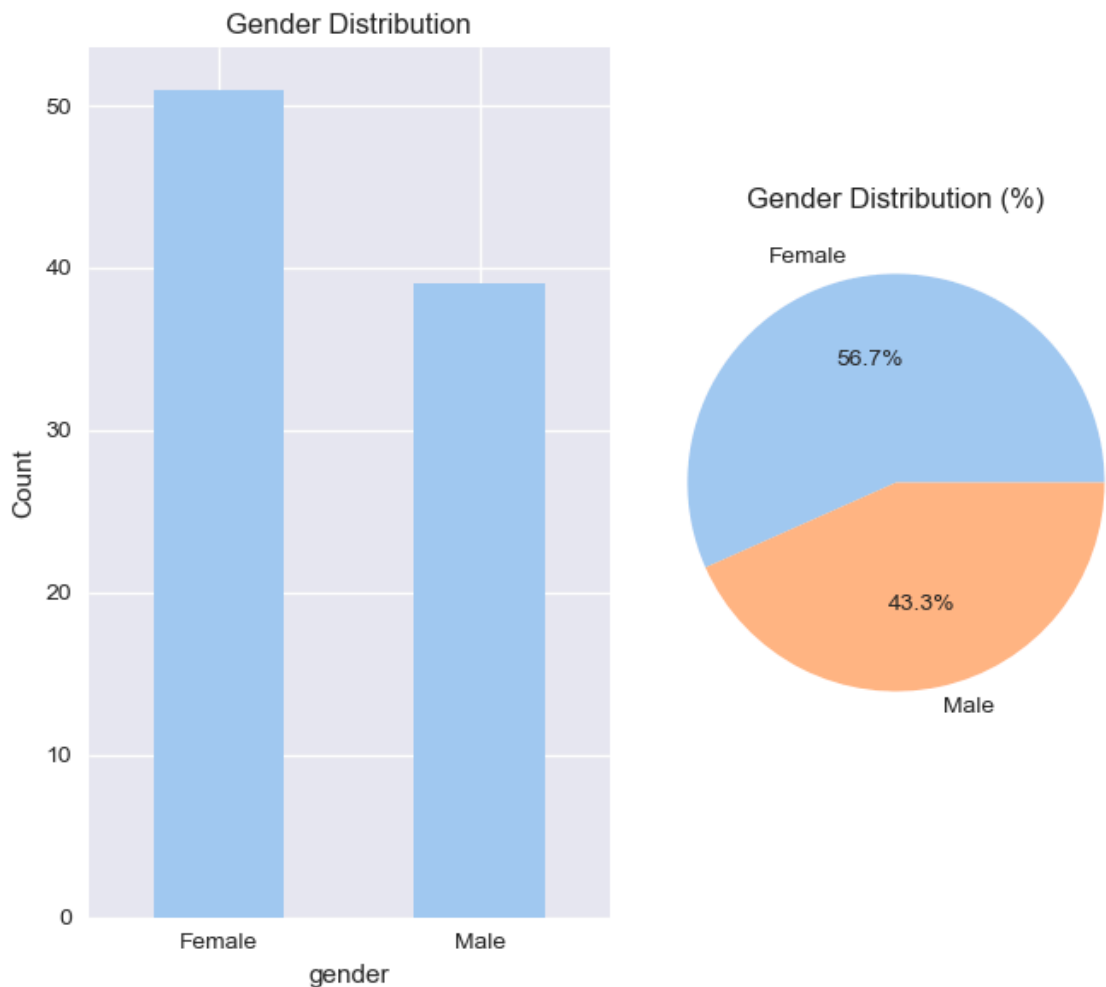
memory usage: 3.5+ KB

None

Statistical Summary:

	height	weight	BMI
count	90.000000	90.000000	90.000000
mean	164.360613	65.271368	24.316047
std	6.843921	11.281900	4.924015
min	144.042039	41.974745	16.134625
25%	160.674183	55.429745	20.546330
50%	164.249366	66.520259	23.766525
75%	167.977460	71.847539	27.321920
max	179.818225	94.558905	42.198310

```
In [30]: plt.figure(figsize=(7, 6))
gender_counts = df_clean['gender'].value_counts()
plt.subplot(1, 2, 1)
gender_counts.plot(kind='bar')
plt.title('Gender Distribution')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.subplot(1, 2, 2)
plt.pie(gender_counts.values, labels=gender_counts.index, autopct='%1.1f%%')
plt.title('Gender Distribution (%)')
plt.tight_layout()
plt.show()
```



```
In [32]: fig, axes = plt.subplots(2, 2, figsize=(15, 12))
fig.suptitle('BMI Analysis Visualizations', fontsize=16)
axes[0,0].hist(df_clean['BMI'], bins=20, alpha=0.7, edgecolor='blue')
axes[0,0].axvline(df_clean['BMI'].mean(), color='red', linestyle='--', label=f'Mean BMI')
axes[0,0].set_title('BMI Distribution')
axes[0,0].set_xlabel('BMI')
axes[0,0].set_ylabel('Frequency')
axes[0,0].legend()
sns.boxplot(data=df_clean, x='gender', y='BMI', ax=axes[0,1])
axes[0,1].set_title('BMI by Gender')
sns.scatterplot(data=df_clean, x='height', y='weight', hue='gender', ax=axes[1,0])
axes[1,0].set_title('Height vs Weight by Gender')
def bmi_category(bmi):
    if bmi < 18.5: return 'UnderWeight'
    elif bmi < 26: return 'Normal'
    elif bmi < 32: return 'Overweight'
    else: return 'Obese'
df_clean['BMI_Category'] = df_clean['BMI'].apply(bmi_category)
category_counts = df_clean['BMI_Category'].value_counts()
axes[1,1].pie(category_counts.values, labels=category_counts.index, autopct='%1.1f%%')
axes[1,1].set_title('BMI Categories Distribution')
plt.tight_layout()
plt.show()
print("Key Inferences:")
print("Average BMI:", f"{df_clean['BMI'].mean():.1f}")
print("BMI range:", f"{df_clean['BMI'].min():.1f} - {df_clean['BMI'].max():.1f}")
print("Gender distribution nearly equal")
print("Males tend to have slightly higher BMI (from boxplot)")
```

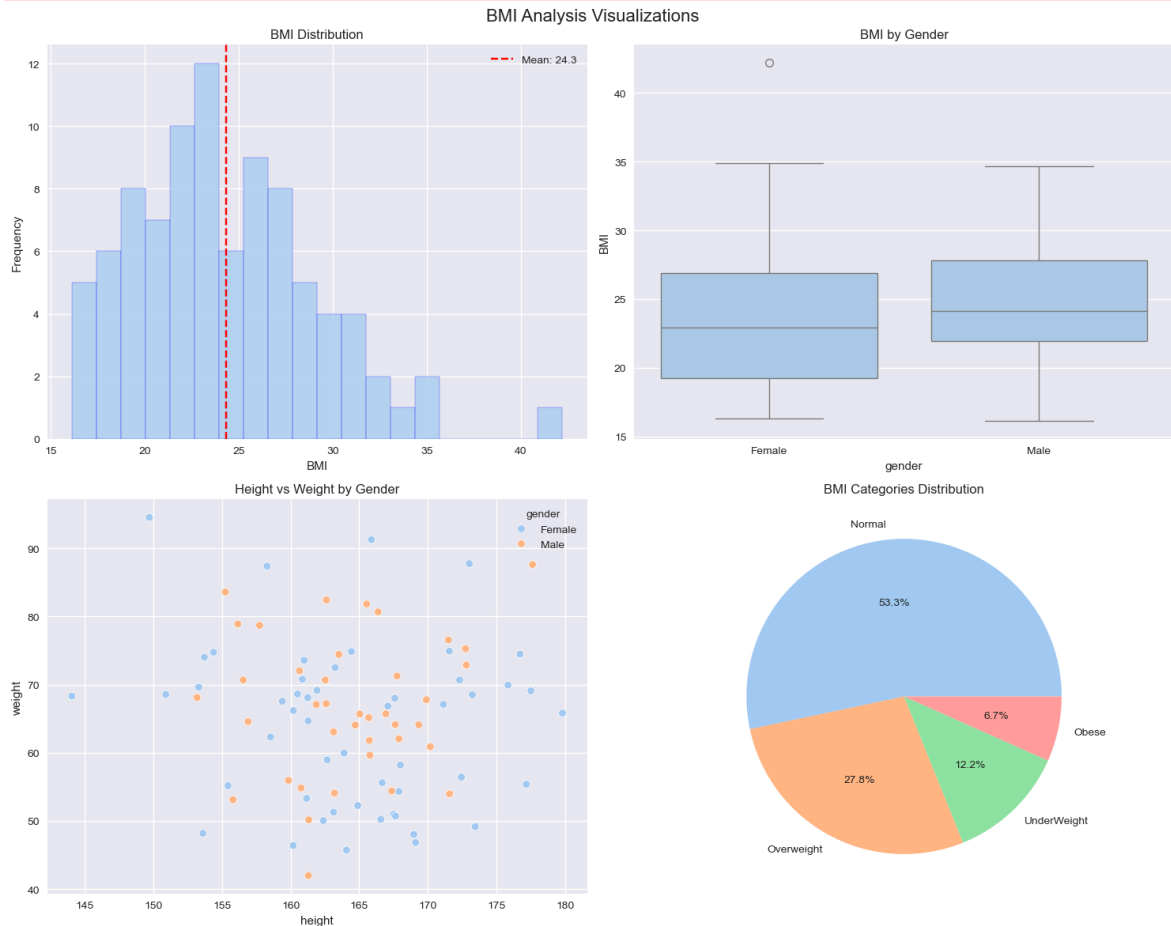

C:\Users\payal\AppData\Local\Temp\ipykernel_33752\1966601202.py:18: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_clean['BMI_Category'] = df_clean['BMI'].apply(bmi_category)
```



Key Inferences:

Average BMI: 24.3

BMI range: 16.1 - 42.2

Gender distribution nearly equal

Males tend to have slightly higher BMI (from boxplot)

```
In [33]: print("Aggregate Functions:")
agg_stats = df_clean[['height', 'weight', 'BMI']].agg(['count', 'mean', 'std', 'min', 'max'])
print(agg_stats.round(2))
```

Aggregate Functions:

	height	weight	BMI
count	90.00	90.00	90.00
mean	164.36	65.27	24.32
std	6.84	11.28	4.92
min	144.04	41.97	16.13
max	179.82	94.56	42.20

```
In [36]: underweight = df_clean[df_clean['BMI'] < 25]
print(f"Entries with BMI < 25 (Underweight):{len(underweight)}")
print("Sample underweight records:")
print(underweight[['height', 'weight', 'BMI', 'gender']].head())
```

Entries with BMI < 25 (Underweight):53

Sample underweight records:

	height	weight	BMI	gender
0	168.973713	48.015551	16.816810	Female
1	163.893886	59.952256	22.319272	Female
2	170.181508	60.887426	21.023396	Male
3	177.184239	55.372673	17.637836	Female
4	163.126773	63.064571	23.699269	Male

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

```

In [42]: #Question 03
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import re
medical_data = {
    'patient_id': [1, 2, 3, 4, 5],
    'diagnosis': ['Flu', 'Cold', 'Diabetes', 'Flu', 'Covid'],
    'visit_cost': [600, 300, 1400, 700, 1000],
    'bp': ['120/80', 'high', '130/85', 'high', '110/70'],
    'sugar_level': [90, np.nan, 180, 110, np.nan]
}
patient_data = {
    'patient_id': [1, 2, 3, 4, 5],
    'name': ['Aryan', None, 'Ritika', 'Joe', None],
    'age': [20, -30, 45, -22, 33],
    'gender': ['M', 'F', 'male', 'female', 'M'],
    'city': ['delhi', 'MUMBAI', 'chennai', 'Delhi', 'mumbai'],
    'admission_date': ['2024/1/5', '2023-12-01', '2024.02.10', '2024/03/15', '2024-4-2'],
    'height_cm': ['170cm', '165', '180cm', 'unknown', '175cm']
}
df1 = pd.DataFrame(medical_data)
df2 = pd.DataFrame(patient_data)
df = pd.merge(df1, df2, on='patient_id', how='inner')
print(df1)
print(df2)
print(df)

```

	patient_id	diagnosis	visit_cost	bp	sugar_level
0	1	Flu	600	120/80	90.0
1	2	Cold	300	high	NaN
2	3	Diabetes	1400	130/85	180.0
3	4	Flu	700	high	110.0
4	5	Covid	1000	110/70	NaN

	patient_id	name	age	gender	city	admission_date	height_cm
0	1	Aryan	20	M	delhi	2024/1/5	170cm
1	2	None	-30	F	MUMBAI	2023-12-01	165
2	3	Ritika	45	male	chennai	2024.02.10	180cm
3	4	Joe	-22	female	Delhi	2024/03/15	unknown
4	5	None	33	M	mumbai	2024-4-2	175cm

	patient_id	diagnosis	visit_cost	bp	sugar_level	name	age	gender	\
0	1	Flu	600	120/80	90.0	Aryan	20	M	
1	2	Cold	300	high	NaN	None	-30	F	
2	3	Diabetes	1400	130/85	180.0	Ritika	45	male	
3	4	Flu	700	high	110.0	Joe	-22	female	
4	5	Covid	1000	110/70	NaN	None	33	M	

	city	admission_date	height_cm
0	delhi	2024/1/5	170cm
1	MUMBAI	2023-12-01	165
2	chennai	2024.02.10	180cm
3	Delhi	2024/03/15	unknown
4	mumbai	2024-4-2	175cm

```

In [43]: df['name'] = df['name'].fillna('unknown')
print("Filling Null Names:")
print(df[['patient_id', 'name']])

```

Filling Null Names:

	patient_id	name
0	1	Aryan
1	2	unknown
2	3	Ritika
3	4	Joe
4	5	unknown

```
In [44]: df['age'] = df['age'].apply(lambda x: abs(x))
print("Fixing Age:")
print(df[['patient_id', 'age']])
```

Fixing Age:

	patient_id	age
0	1	20
1	2	30
2	3	45
3	4	22
4	5	33

```
In [45]: df['age'] = df['age'].apply(lambda x: abs(x))
print("After Fixing Age:")
print(df[['patient_id', 'age']])
```

After Fixing Age:

	patient_id	age
0	1	20
1	2	30
2	3	45
3	4	22
4	5	33

```
In [46]: df['gender'] = df['gender'].replace({
    'M': 'Male', 'F': 'Female',
    'male': 'Male', 'female': 'Female'
})
print("After Standardizing Gender:")
print(df[['patient_id', 'gender']])
```

After Standardizing Gender:

	patient_id	gender
0	1	Male
1	2	Female
2	3	Male
3	4	Female
4	5	Male

```
In [47]: def format_date(date):
    if pd.isnull(date):
        return date
    parts = re.findall(r'\d+', str(date))
    if len(parts) == 3:
        return f"{parts[0]}-{parts[1].zfill(2)}-{parts[2].zfill(2)}"
    return date
df['admission_date'] = df['admission_date'].apply(format_date)
print("After Formatting Dates:")
print(df[['patient_id', 'admission_date']])
```

After Formatting Dates:

	patient_id	admission_date
0	1	2024-01-05
1	2	2023-12-01
2	3	2024-02-10
3	4	2024-03-15
4	5	2024-04-02

```
In [48]: df['height_cm'] = df['height_cm'].astype(str).str.extract('(\d+\.?*\d*)')
df['height_cm'] = pd.to_numeric(df['height_cm'], errors='coerce')
df['height_cm'] = df['height_cm'].fillna(df['height_cm'].mean())
print("After Cleaning Height:")
print(df[['patient_id', 'height_cm']])
```

After Cleaning Height:

	patient_id	height_cm
0	1	170.0
1	2	165.0
2	3	180.0
3	4	172.5
4	5	175.0

```
In [49]: df['sugar_level'] = df['sugar_level'].fillna(df['sugar_level'].mean())
print("After Filling Sugar Level:")
print(df[['patient_id', 'sugar_level']])
```

After Filling Sugar Level:

	patient_id	sugar_level
0	1	90.000000
1	2	126.666667
2	3	180.000000
3	4	110.000000
4	5	126.666667

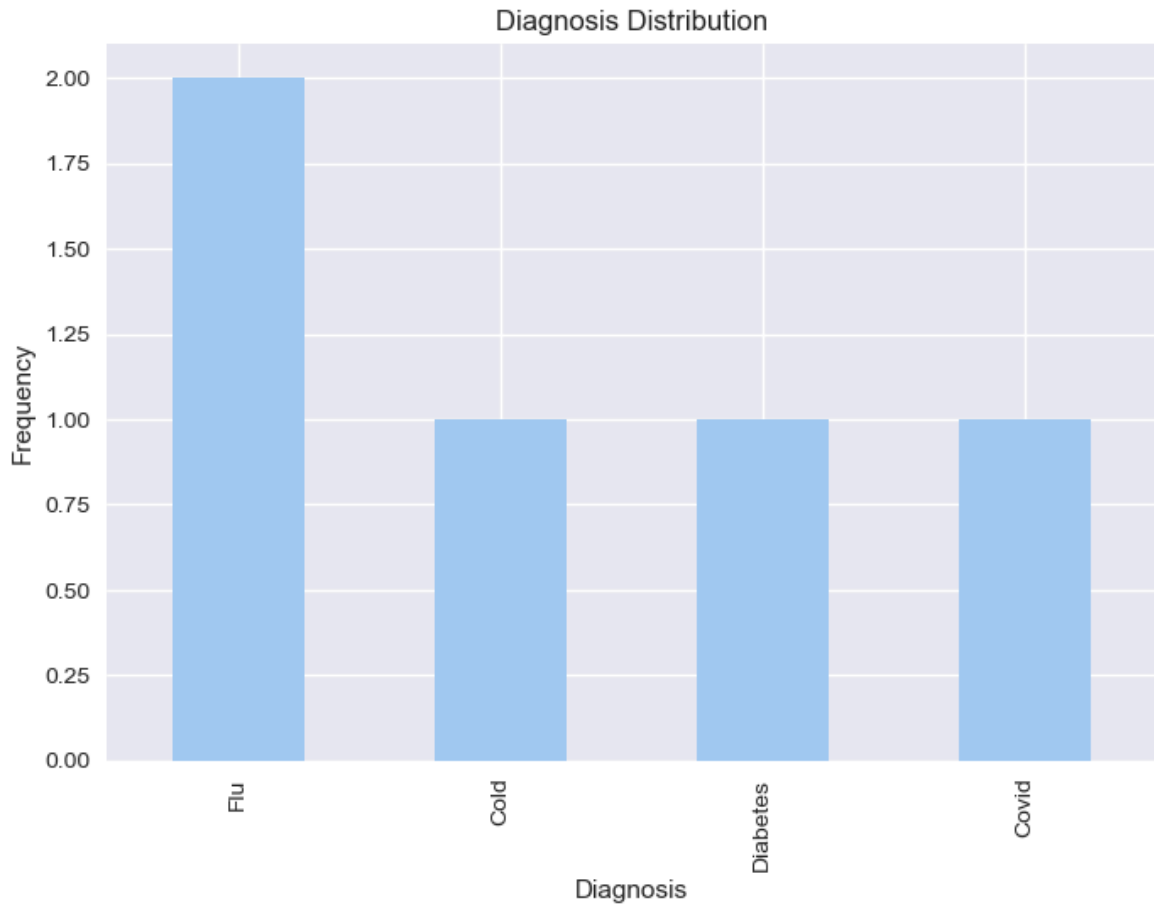
```
In [50]: grouped = df.groupby('gender')['visit_cost'].agg(['mean', 'min', 'max'])
print("Visit Cost Stats by Gender:")
print(grouped)
```

Visit Cost Stats by Gender:

	mean	min	max
gender			
Female	500.0	300	700
Male	1000.0	600	1400

```
In [51]: print("Plotting Diagnosis Histogram")
plt.figure()
df['diagnosis'].value_counts().plot(kind='bar')
plt.xlabel('Diagnosis')
plt.ylabel('Frequency')
plt.title('Diagnosis Distribution')
plt.show()
```

Plotting Diagnosis Histogram



```
In [52]: print("Final Cleaned Dataset:")
         print(df)
```

Final Cleaned Dataset:

	patient_id	diagnosis	visit_cost	bp	sugar_level	name	age	\
0	1	Flu	600	120/80	90.000000	Aryan	20	
1	2	Cold	300	high	126.666667	unknown	30	
2	3	Diabetes	1400	130/85	180.000000	Ritika	45	
3	4	Flu	700	high	110.000000	Joe	22	
4	5	Covid	1000	110/70	126.666667	unknown	33	

	gender	city	admission_date	height_cm
0	Male	delhi	2024-01-05	170.0
1	Female	MUMBAI	2023-12-01	165.0
2	Male	chennai	2024-02-10	180.0
3	Female	Delhi	2024-03-15	172.5
4	Male	mumbai	2024-04-02	175.0

In []:

In []:

In []:

In []:

In []:

In []:

In []:

```
In [53]: # Question 04
def dfs(graph, start, goal, visited=None, path=None):
    if visited is None:
        visited = set()
    if path is None:
        path = []
    visited.add(start)
    path.append(start)
    if start == goal:
        return path
    for neighbor in graph[start]:
        if neighbor not in visited:
            result = dfs(graph, neighbor, goal, visited, path.copy())
            if result:
                return result
    return None
graph_static = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E', 'G'],
    'C': ['A', 'D', 'H'],
    'D': ['B', 'C', 'E', 'F'],
    'E': ['B', 'D', 'F'],
    'F': ['C', 'D', 'G', 'B']
}
print("\nStatic Graph:", graph_static)
```

Static Graph: {'A': ['B', 'C'], 'B': ['A', 'D', 'E', 'G'], 'C': ['A', 'D', 'H'], 'D': ['B', 'C', 'E', 'F'], 'E': ['B', 'D', 'F'], 'F': ['C', 'D', 'G', 'B']}

```
In [1]: graph_dynamic = {}
nodes = int(input("\nEnter number of nodes: "))
edges = int(input("Enter number of edges: "))
print("\nEnter node names:")
node_list = []
for i in range(nodes):
    n = input(f"Node {i+1}: ")
    graph_dynamic[n] = []
    node_list.append(n)
print("\nEnter edges (format: node1 node2):")
for i in range(edges):
    u, v = input(f"Edge {i+1}: ").split()
    graph_dynamic[u].append(v)
    graph_dynamic[v].append(u)
print("\nDynamic Graph:", graph_dynamic)
```

Enter node names:

Enter edges (format: node1 node2):

Dynamic Graph: {'A': ['B', 'D', 'C'], 'B': ['A'], 'C': ['A'], 'D': ['A']}

```
In [60]: start2 = input("\nEnter START node for dynamic graph: ")
goal2 = input("Enter GOAL node for dynamic graph: ")
path2 = dfs(graph_dynamic, start2, goal2)
```

```
In [62]: if path2:
    print("Path in Dynamic Graph:", " to ".join(path2))
else:
    print("No path found in dynamic graph")
```

No path found in dynamic graph

```
In [64]: #Question 05
from collections import deque, defaultdict
def bfs(graph, start, goal):
    visited = set()
    queue = deque([start])
    parent = {start: None}
    traversal = []
    while queue:
        node = queue.popleft()
        visited.add(node)
        traversal.append(node)
        if node == goal:
            break
        for neighbor in graph[node]:
            if neighbor not in visited:
                queue.append(neighbor)
                parent[neighbor] = node
    path = []
    if goal in parent:
        current = goal
        while current:
            path.append(current)
            current = parent[current]
        path.reverse()
    return traversal, path
def print_graph(graph):
    print("Graph:")
    for node in sorted(graph):
        print(f"{node}: {graph[node]}")
graph1 = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E', 'F'],
    'C': ['A', 'E', 'F'],
    'D': ['B', 'C', 'E', 'F'],
    'E': ['B', 'D', 'F'],
    'F': ['B', 'C', 'D', 'E']
}
print_graph(graph1)
start1 = input("\nStart node: ").upper()
goal1 = input("Goal node: ").upper()
order1, path1 = bfs(graph1, start1, goal1)
print(f"Traversal: {' → '.join(order1)}")
if path1:
    print(f"Path: {' → '.join(path1)}")
else:
    print("No path found")
```

Graph:

A: ['B', 'C']

B: ['A', 'D', 'E', 'F']

C: ['A', 'E', 'F']

D: ['B', 'C', 'E', 'F']

E: ['B', 'D', 'F']

F: ['B', 'C', 'D', 'E']

Traversal: A → B → C → D → E → F

Path: A → B → D → E → F


```

In [5]: from collections import defaultdict

# Define print_graph function
def print_graph(graph):
    print("\nGraph representation:")
    for node in graph:
        print(f"{node}: {' '.join(graph[node])}")

# Define the bfs function
def bfs(graph, start, goal):
    visited = set([start])
    queue = [start]
    order = []
    parent = {start: None}

    while queue:
        node = queue.pop(0)
        order.append(node)

        if node == goal:
            # Reconstruct path
            path = []
            while node:
                path.append(node)
                node = parent.get(node)
            return order, path[::-1]

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
                parent[neighbor] = node

    return order, [] # No path found

n_nodes = int(input("Nodes: "))
n_edges = int(input("Edges: "))
graph2 = defaultdict(list)
print("Enter nodes:")
nodes = [input(f"Node {i+1}: ").upper() for i in range(n_nodes)]
print("Enter edges (u v):")
for _ in range(n_edges):
    u, v = input().split()
    u, v = u.upper(), v.upper()
    graph2[u].append(v)
    graph2[v].append(u)
print_graph(graph2)
start2 = input("\nStart node: ").upper()
goal2 = input("Goal node: ").upper()
order2, path2 = bfs(graph2, start2, goal2)
print(f"Traversal: {' → '.join(order2)}")
if path2:
    print(f"Path: {' → '.join(path2)}")
else:
    print("No path found")

```

Enter nodes:

Enter edges (u v):

Graph representation:

C: D

D: C

E: F

F: E

G: H

H: G

Traversal: C → D

No path found

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

```

In [9]: #Question 06
import heapq
def a_star(graph, heuristic, start, goal):
    print("\n[INFO] Starting A* Algorithm")
    print("Start Node:", start)
    print("Goal Node:", goal)
    open_list = []
    heapq.heappush(open_list, (heuristic[start], start))
    g_cost = {node: float('inf') for node in graph}
    g_cost[start] = 0
    parent = {node: None for node in graph}
    closed_list = set()
    expansion_order = []
    print("Initial Open List:", open_list)
    while open_list:
        f, current = heapq.heappop(open_list)
        print("Expanding Node:", current)
        print("f(n) =", f)
        expansion_order.append(current)
        closed_list.add(current)
        print("Closed List:", closed_list)
        for neighbor, cost in graph[current].items():
            print(f"Neighbor: {neighbor}, Cost: {cost}")
            if neighbor in closed_list:
                print("Already visited")
                continue
            tentative_g = g_cost[current] + cost
            print(f" [CALC] g({neighbor}) = {tentative_g}")
            if tentative_g < g_cost[neighbor]:
                print(f"Updating {neighbor}")
                g_cost[neighbor] = tentative_g
                parent[neighbor] = current
                f_cost = tentative_g + heuristic[neighbor]
                heapq.heappush(open_list, (f_cost, neighbor))
                print(f"Added with f(n) = {f_cost}")
        print("Open List:", open_list)
    path = []
    node = goal
    while node:
        path.append(node)
        node = parent[node]
    path.reverse()
    graph = {
        'E': {'B': 2, 'C': 3},
        'F': {'A': 1, 'C': 3, 'E': 6},
        'G': {'A': 3, 'D': 5},
        'H': {'B': 3, 'C': 1, 'E': 2},
        'I': {'B': 6, 'D': 4}
    }
    heuristic = {
        'E': 7,
        'F': 6,
        'G': 5,
        'H': 4,
        'I': 3
    }

```

```

}
print("Graph:")
for node in graph:
    print(node, "->", graph[node])

```

Graph:

```

E -> {'B': 2, 'C': 3}
F -> {'A': 1, 'C': 3, 'E': 6}
G -> {'A': 3, 'D': 5}
H -> {'B': 3, 'C': 1, 'E': 2}
I -> {'B': 6, 'D': 4}

```

```

In [10]: print("Heuristic Values:")
        for node in heuristic:
            print(f"h({node}) =", heuristic[node])

```

Heuristic Values:

```

h(E) = 7
h(F) = 6
h(G) = 5
h(H) = 4
h(I) = 3

```

```

In [16]: graph = {
        'A': {'B': 1, 'C': 4},
        'B': {'A': 1, 'D': 2, 'E': 5},
        'C': {'A': 4, 'F': 2},
        'D': {'B': 2, 'G': 3},
        'E': {'B': 5, 'H': 1},
        'F': {'C': 2, 'H': 4},
        'G': {'D': 3, 'H': 2},
        'H': {'E': 1, 'F': 4, 'G': 2}
    }

    heuristic = {
        'A': 7,
        'B': 6,
        'C': 4,
        'D': 5,
        'E': 2,
        'F': 3,
        'G': 1,
        'H': 0
    }

    start = input("\nEnter start node: ").upper()
    goal = 'H'
    if start in graph:
        a_star(graph, heuristic, start, goal)
    else:
        print(f"Error: Node '{start}' does not exist in the graph. Available nodes:

```

```
[INFO] Starting A* Algorithm
Start Node: B
Goal Node: H
Initial Open List: [(6, 'B')]
Expanding Node: B
f(n) = 6
Closed List: {'B'}
Neighbor: A, Cost: 1
  [CALC] g(A) = 1
Updating A
Added with f(n) = 8
Neighbor: D, Cost: 2
  [CALC] g(D) = 2
Updating D
Added with f(n) = 7
Neighbor: E, Cost: 5
  [CALC] g(E) = 5
Updating E
Added with f(n) = 7
Open List: [(7, 'D'), (8, 'A'), (7, 'E')]
Expanding Node: D
f(n) = 7
Closed List: {'D', 'B'}
Neighbor: B, Cost: 2
Already visited
Neighbor: G, Cost: 3
  [CALC] g(G) = 5
Updating G
Added with f(n) = 6
Open List: [(6, 'G'), (8, 'A'), (7, 'E')]
Expanding Node: G
f(n) = 6
Closed List: {'D', 'B', 'G'}
Neighbor: D, Cost: 3
Already visited
Neighbor: H, Cost: 2
  [CALC] g(H) = 7
Updating H
Added with f(n) = 7
Open List: [(7, 'E'), (8, 'A'), (7, 'H')]
Expanding Node: E
f(n) = 7
Closed List: {'D', 'E', 'B', 'G'}
Neighbor: B, Cost: 5
Already visited
Neighbor: H, Cost: 1
  [CALC] g(H) = 6
Updating H
Added with f(n) = 6
Open List: [(6, 'H'), (8, 'A'), (7, 'H')]
Expanding Node: H
f(n) = 6
Closed List: {'D', 'B', 'G', 'H', 'E'}
Neighbor: E, Cost: 1
Already visited
Neighbor: F, Cost: 4
  [CALC] g(F) = 10
Updating F
Added with f(n) = 13
Neighbor: G, Cost: 2
```

```

Already visited
Open List: [(7, 'H'), (8, 'A'), (13, 'F')]
Expanding Node: H
f(n) = 7
Closed List: {'D', 'B', 'G', 'H', 'E'}
Neighbor: E, Cost: 1
Already visited
Neighbor: F, Cost: 4
  [CALC] g(F) = 10
Neighbor: G, Cost: 2
Already visited
Open List: [(8, 'A'), (13, 'F')]
Expanding Node: A
f(n) = 8
Closed List: {'D', 'B', 'G', 'H', 'A', 'E'}
Neighbor: B, Cost: 1
Already visited
Neighbor: C, Cost: 4
  [CALC] g(C) = 5
Updating C
Added with f(n) = 9
Open List: [(9, 'C'), (13, 'F')]
Expanding Node: C
f(n) = 9
Closed List: {'D', 'B', 'G', 'C', 'H', 'A', 'E'}
Neighbor: A, Cost: 4
Already visited
Neighbor: F, Cost: 2
  [CALC] g(F) = 7
Updating F
Added with f(n) = 10
Open List: [(10, 'F'), (13, 'F')]
Expanding Node: F
f(n) = 10
Closed List: {'D', 'B', 'G', 'C', 'H', 'A', 'E', 'F'}
Neighbor: C, Cost: 2
Already visited
Neighbor: H, Cost: 4
Already visited
Open List: [(13, 'F')]
Expanding Node: F
f(n) = 13
Closed List: {'D', 'B', 'G', 'C', 'H', 'A', 'E', 'F'}
Neighbor: C, Cost: 2
Already visited
Neighbor: H, Cost: 4
Already visited
Open List: []

```

In []:

In []:

In []:

In []:

In []:

```
In [18]: #Question 07
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler, StandardScaler
data = {
    'Age': [20, 45, 33, 50, 23, 60, 45, 30, 65, 28],
    'Weight': [55, 70, 65, 80, 50, 90, 75, 60, 85, 58],
    'Blood_Pressure': [120, 140, 125, 145, 115, 150, 135, 112, 144, 108],
    'Cholesterol': [180, 220, 200, 240, 170, 260, 230, 190, 250, 175],
    'Sugar_Level': [90, 110, 100, 130, 85, 140, 120, 95, 135, 88]
}

df = pd.DataFrame(data)
print(df)
print(df.head())
```

	Age	Weight	Blood_Pressure	Cholesterol	Sugar_Level
0	20	55	120	180	90
1	45	70	140	220	110
2	33	65	125	200	100
3	50	80	145	240	130
4	23	50	115	170	85
5	60	90	150	260	140
6	45	75	135	230	120
7	30	60	112	190	95
8	65	85	144	250	135
9	28	58	108	175	88

	Age	Weight	Blood_Pressure	Cholesterol	Sugar_Level
0	20	55	120	180	90
1	45	70	140	220	110
2	33	65	125	200	100
3	50	80	145	240	130
4	23	50	115	170	85

```
In [19]: minmax = MinMaxScaler()
minmax_scaled = minmax.fit_transform(df)
minmax_df = pd.DataFrame(minmax_scaled, columns=df.columns)
print(minmax_df.head())
```

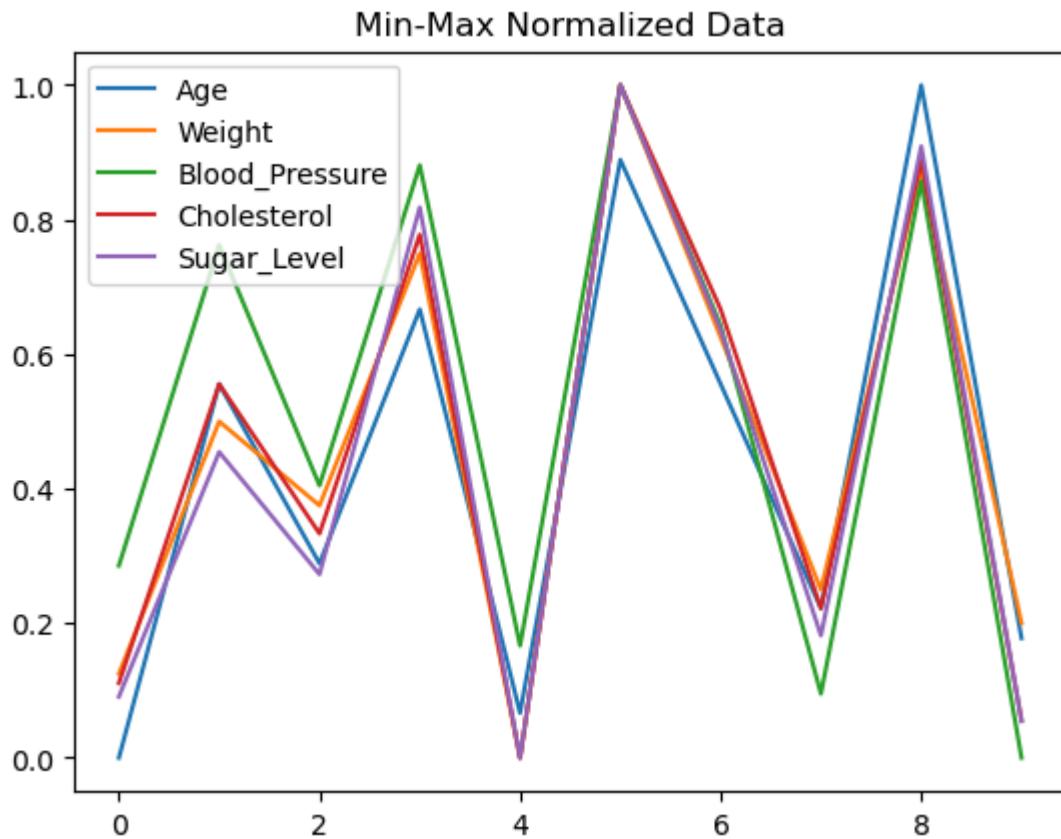
	Age	Weight	Blood_Pressure	Cholesterol	Sugar_Level
0	0.000000	0.125	0.285714	0.111111	0.090909
1	0.555556	0.500	0.761905	0.555556	0.454545
2	0.288889	0.375	0.404762	0.333333	0.272727
3	0.666667	0.750	0.880952	0.777778	0.818182
4	0.066667	0.000	0.166667	0.000000	0.000000

```
In [20]: standard = StandardScaler()
standard_scaled = standard.fit_transform(df)
standard_df = pd.DataFrame(standard_scaled, columns=df.columns)
print(standard_df.head())
```

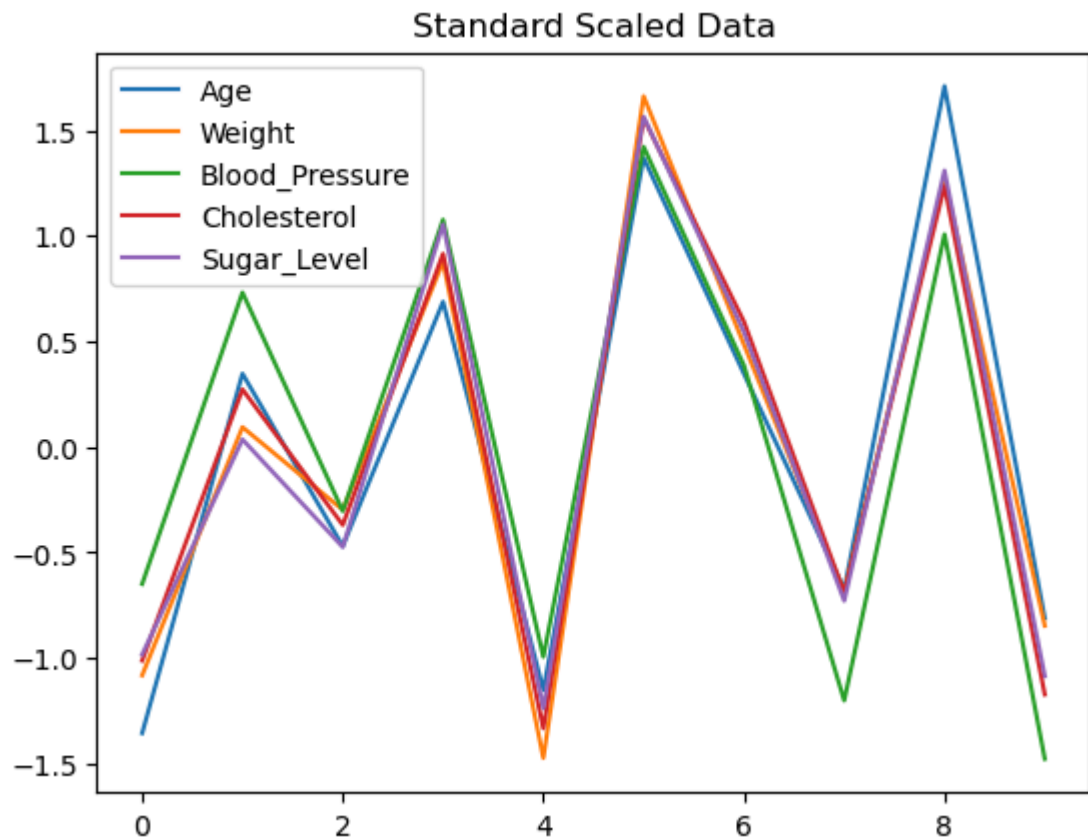
	Age	Weight	Blood_Pressure	Cholesterol	Sugar_Level
0	-1.354996	-1.081032	-0.6486	-1.011274	-0.983096
1	0.347260	0.094003	0.7314	0.272883	0.035656
2	-0.469823	-0.297675	-0.3036	-0.369195	-0.473720
3	0.687712	0.877359	1.0764	0.914962	1.054408
4	-1.150725	-1.472710	-0.9936	-1.332313	-1.237784

```
In [21]: plt.figure()
for column in minmax_df.columns:
    plt.plot(minmax_df[column], label=column)
```

```
plt.title("Min-Max Normalized Data")  
plt.legend()  
plt.show()
```



```
In [22]: plt.figure()  
for column in standard_df.columns:  
    plt.plot(standard_df[column], label=column)  
plt.title("Standard Scaled Data")  
plt.legend()  
plt.show()
```

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

```
In [25]: #QUESTION 8
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
np.random.seed(42)
data = {
    'Age': np.random.randint(20, 70, 200),
    'Weight': np.random.randint(50, 100, 200),
    'Blood_Pressure': np.random.randint(110, 160, 200),
    'Cholesterol': np.random.randint(150, 300, 200),
    'Sugar_Level': np.random.randint(70, 150, 200),
    'Heart_Rate': np.random.randint(60, 100, 200)
}
df = pd.DataFrame(data)
df['Health_Score'] = (
    0.3 * df['Age'] +
    0.2 * df['Weight'] +
    0.2 * df['Blood_Pressure'] +
    0.1 * df['Cholesterol'] +
    0.1 * df['Sugar_Level'] +
    0.1 * df['Heart_Rate']
)
print(df.head())
X = df.drop('Health_Score', axis=1)
y = df['Health_Score']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

	Age	Weight	Blood_Pressure	Cholesterol	Sugar_Level	Heart_Rate	\
0	58	64	157	211	138	63	
1	48	81	148	233	103	84	
2	34	81	138	261	75	84	
3	62	73	151	235	122	77	
4	27	90	135	290	135	99	

	Health_Score
0	102.8
1	102.2
2	96.0
3	106.8
4	105.5

```
In [26]: lr = LinearRegression()
sfs_forward_lr = SequentialFeatureSelector(lr, n_features_to_select=3, direction='forward')
sfs_forward_lr.fit(X_train, y_train)
forward_lr_features = X.columns[sfs_forward_lr.get_support()]
print("[Forward Selected Features]:", list(forward_lr_features))
knn = KNeighborsRegressor(n_neighbors=5)
sfs_forward_knn = SequentialFeatureSelector(knn, n_features_to_select=3, direction='forward')
sfs_forward_knn.fit(X_train, y_train)
forward_knn_features = X.columns[sfs_forward_knn.get_support()]
print("[kNN - Forward Selected Features]:", list(forward_knn_features))
```

```
[Forward Selected Features]: ['Age', 'Blood_Pressure', 'Cholesterol']
[kNN - Forward Selected Features]: ['Age', 'Blood_Pressure', 'Cholesterol']
```

```
In [27]: sfs_backward_lr = SequentialFeatureSelector(lr, n_features_to_select=3, direction='backward')
sfs_backward_lr.fit(X_train, y_train)
backward_lr_features = X.columns[sfs_backward_lr.get_support()]
print("[Backward Selected Features]:", list(backward_lr_features))
sfs_backward_knn = SequentialFeatureSelector(knn, n_features_to_select=3, direction='backward')
sfs_backward_knn.fit(X_train, y_train)
backward_knn_features = X.columns[sfs_backward_knn.get_support()]
print("[kNN - Backward Selected Features]:", list(backward_knn_features))
```

[Backward Selected Features]: ['Age', 'Blood_Pressure', 'Cholesterol']

[kNN - Backward Selected Features]: ['Age', 'Blood_Pressure', 'Cholesterol']

```
In [28]: def evaluate_model(model, features):
model.fit(X_train[features], y_train)
pred = model.predict(X_test[features])
return r2_score(y_test, pred)
print("[Forward R2]:", evaluate_model(lr, forward_lr_features))
print("[Backward R2]:", evaluate_model(lr, backward_lr_features))
print("[kNN Forward R2]:", evaluate_model(knn, forward_knn_features))
print("[kNN Backward R2]:", evaluate_model(knn, backward_knn_features))
```

[Forward R2]: 0.6912607457724744

[Backward R2]: 0.6912607457724744

[kNN Forward R2]: 0.672494141965617

[kNN Backward R2]: 0.672494141965617

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

```

In [32]: #Question 09
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.datasets import make_classification
plt.style.use('seaborn-v0_8')
sns.set_palette("husl")
np.random.seed(42)
X, y = make_classification(n_samples=100, n_features=20, n_informative=15,
                           n_redundant=5, n_clusters_per_class=1, random_state=42)

features = [f'Sensor_{i+1}' for i in range(20)]
df = pd.DataFrame(X, columns=features)
df['cluster'] = y
# Save dataset
df.to_csv('smart_city_monitor.csv', index=False)
print("Dataset loaded:")
print(f"Shape: {df.shape} (100 samples, 30 features)")
print(df.head())

```

Dataset loaded:

Shape: (100, 21) (100 samples, 30 features)

	Sensor_1	Sensor_2	Sensor_3	Sensor_4	Sensor_5	Sensor_6	Sensor_7	\
0	2.705491	1.907507	2.172789	3.268840	-4.684966	1.623045	5.479531	
1	1.127386	2.132249	2.211492	2.656430	-2.328380	-6.318219	-1.719949	
2	-2.520771	-5.774444	-0.361292	0.871057	-14.006996	2.615716	9.350641	
3	1.565380	2.603050	0.750137	-0.657664	3.945336	0.032244	-5.198501	
4	1.589424	0.399229	-4.108595	0.173939	3.486327	2.886338	1.535123	

	Sensor_8	Sensor_9	Sensor_10	...	Sensor_12	Sensor_13	Sensor_14	\
0	-3.614347	0.837689	1.516202	...	2.130041	0.104207	-2.563779	
1	-1.099374	0.379129	-3.068377	...	1.547615	-0.200332	1.521392	
2	3.208690	1.583894	-1.929346	...	-5.214279	2.858990	0.577157	
3	-6.003980	-0.241763	1.415434	...	4.052248	-1.962198	-0.122050	
4	-2.075570	-3.271679	-1.768313	...	4.104246	-2.746981	1.852727	

	Sensor_15	Sensor_16	Sensor_17	Sensor_18	Sensor_19	Sensor_20	cluster
0	-0.085493	-0.373789	-0.880643	-2.389128	-3.155991	1.665411	0
1	0.417144	4.027735	-1.269214	3.924564	1.165188	1.982787	1
2	2.993234	1.847510	1.021059	-3.095966	-2.724207	4.304583	0
3	-3.242297	1.583910	-1.886235	-0.583814	-1.362431	-1.619260	1
4	-0.905695	1.094603	2.006458	-1.214994	1.149798	2.048901	1

[5 rows x 21 columns]

```

In [33]: scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[features])
print("Data standardized (mean=0, std=1)")
print("Scaled data stats:")
print(pd.DataFrame(X_scaled, columns=features).describe().round(3))

```

Data standardized (mean=0, std=1)

Scaled data stats:

	Sensor_1	Sensor_2	Sensor_3	Sensor_4	Sensor_5	Sensor_6	Sensor_7	\
count	100.000	100.000	100.000	100.000	100.000	100.000	100.000	
mean	0.000	0.000	-0.000	-0.000	-0.000	-0.000	-0.000	
std	1.005	1.005	1.005	1.005	1.005	1.005	1.005	
min	-3.358	-2.436	-3.087	-2.045	-2.234	-2.465	-2.293	
25%	-0.648	-0.655	-0.600	-0.695	-0.622	-0.783	-0.725	
50%	0.067	0.135	0.137	0.021	0.021	0.151	0.020	
75%	0.571	0.775	0.625	0.749	0.639	0.680	0.611	
max	2.614	2.282	2.627	2.565	2.058	2.087	2.341	

	Sensor_8	Sensor_9	Sensor_10	Sensor_11	Sensor_12	Sensor_13	\
count	100.000	100.000	100.000	100.000	100.000	100.000	
mean	0.000	-0.000	-0.000	0.000	-0.000	0.000	
std	1.005	1.005	1.005	1.005	1.005	1.005	
min	-2.241	-2.350	-2.254	-3.036	-2.417	-2.568	
25%	-0.742	-0.644	-0.720	-0.541	-0.797	-0.590	
50%	-0.038	-0.111	0.065	0.131	-0.149	-0.033	
75%	0.590	0.704	0.746	0.687	0.506	0.657	
max	2.277	2.922	2.430	1.859	2.426	2.208	

	Sensor_14	Sensor_15	Sensor_16	Sensor_17	Sensor_18	Sensor_19	\
count	100.000	100.000	100.000	100.000	100.000	100.000	
mean	0.000	-0.000	0.000	-0.000	-0.000	-0.000	
std	1.005	1.005	1.005	1.005	1.005	1.005	
min	-2.480	-2.981	-2.194	-2.117	-2.216	-1.892	
25%	-0.731	-0.636	-0.650	-0.685	-0.800	-0.823	
50%	-0.031	-0.075	0.033	0.060	-0.073	-0.019	
75%	0.810	0.532	0.684	0.513	0.738	0.604	
max	2.411	2.737	2.365	2.805	2.310	2.412	

	Sensor_20
count	100.000
mean	-0.000
std	1.005
min	-2.617
25%	-0.701
50%	0.035
75%	0.646
max	1.914

```
In [34]: pca = PCA()
X_pca = pca.fit_transform(X_scaled)
print("PCA applied (30 components)")
```

PCA applied (30 components)

```
In [35]: explained_var = pca.explained_variance_ratio_
cumulative_var = np.cumsum(explained_var)
print("Components Variance % Cumulative %")
for i in range(10):
    print(f"PC{i+1:2d}|{explained_var[i]*100:7.2f}|{cumulative_var[i]*100:10.2f}")
n_95 = np.argmax(cumulative_var >= 0.95) + 1
print(f"{n_95} components retain 95% variance")
print(f"components retain {cumulative_var[4]*100:.1f}% variance")
```

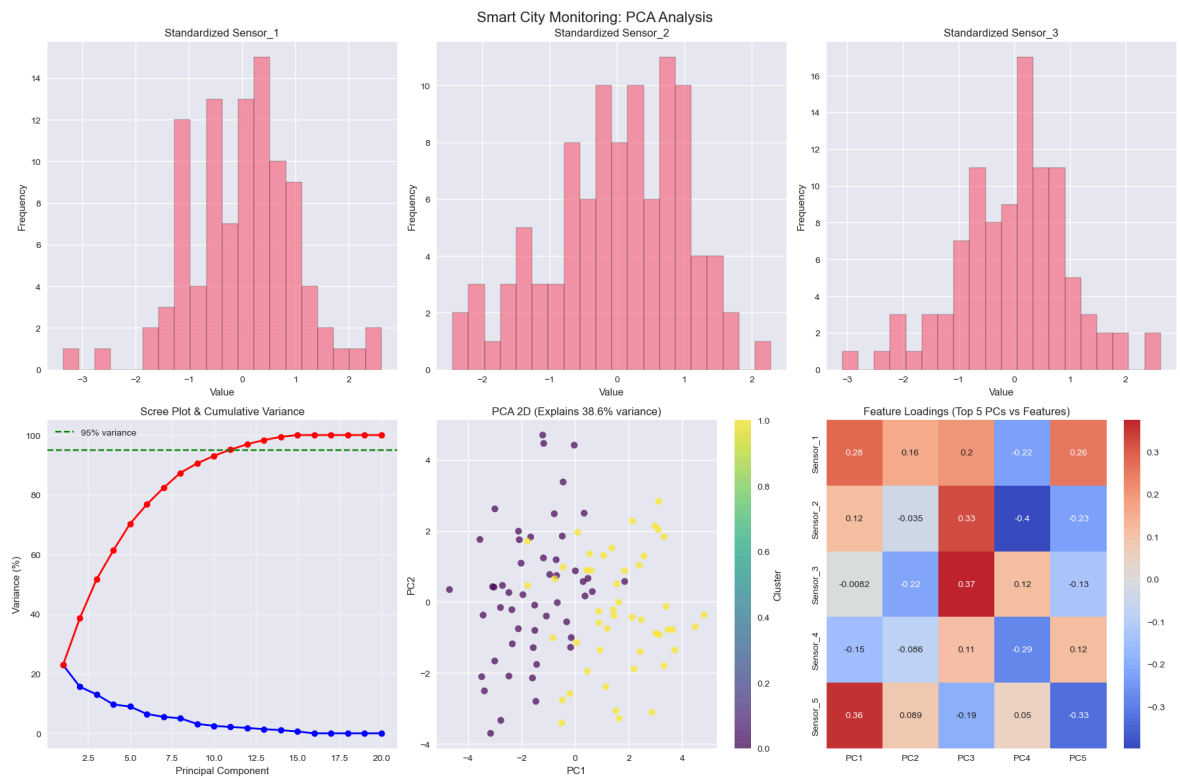
Components	Variance %	Cumulative %
PC 1	22.92	22.92
PC 2	15.64	38.56
PC 3	13.03	51.59
PC 4	9.69	61.28
PC 5	8.97	70.25
PC 6	6.50	76.75
PC 7	5.54	82.28
PC 8	5.02	87.31
PC 9	3.17	90.48
PC10	2.51	92.99

11 components retain 95% variance
components retain 70.3% variance

```
In [36]: pca_2d = PCA(n_components=2)
X_pca_2d = pca_2d.fit_transform(X_scaled)
print("Dataset transformed to 2D (PC1, PC2)")
print(f"PC1 variance: {pca_2d.explained_variance_ratio_[0]*100:.1f}%")
print(f"PC2 variance: {pca_2d.explained_variance_ratio_[1]*100:.1f}%")
```

Dataset transformed to 2D (PC1, PC2)
PC1 variance: 22.9%
PC2 variance: 15.6%

```
In [45]: fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('Smart City Monitoring: PCA Analysis', fontsize=16)
for i, feat in enumerate(features[:3]):
    axes[0, i].hist(X_scaled[:, i], bins=20, alpha=0.7, edgecolor='black')
    axes[0, i].set_title(f'Standardized {feat}')
    axes[0, i].set_xlabel('Value')
    axes[0, i].set_ylabel('Frequency')
axes[1, 0].plot(range(1, 21), explained_var * 100, 'bo-', linewidth=2)
axes[1, 0].plot(range(1, 21), cumulative_var * 100, 'ro-', linewidth=2)
axes[1, 0].axhline(y=95, color='g', linestyle='--', label='95% variance')
axes[1, 0].set_title('Scree Plot & Cumulative Variance')
axes[1, 0].set_xlabel('Principal Component')
axes[1, 0].set_ylabel('Variance (%)')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)
scatter = axes[1, 1].scatter(X_pca_2d[:, 0], X_pca_2d[:, 1], c=df['cluster'],
                             cmap='viridis', alpha=0.7, s=50)
axes[1, 1].set_title(f'PCA 2D (Explains {pca_2d.explained_variance_ratio_.sum()}% variance)')
axes[1, 1].set_xlabel('PC1')
axes[1, 1].set_ylabel('PC2')
plt.colorbar(scatter, ax=axes[1, 1], label='Cluster')
loadings = pca.components_[:5, :5].T
sns.heatmap(loadings, annot=True, cmap='coolwarm', center=0,
             xticklabels=[f'PC{i+1}' for i in range(5)],
             yticklabels=features[:5], ax=axes[1, 2])
axes[1, 2].set_title('Feature Loadings (Top 5 PCs vs Features)')
plt.tight_layout()
plt.show()
```



In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

```

In [48]: #Question 10
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
import warnings
warnings.filterwarnings('ignore')
plt.style.use('seaborn-v0_8')
sns.set_palette("husl")
np.random.seed(42)
n = 100
years_exp = np.random.uniform(1, 20, n)
salary = 30000 + 2000 * years_exp + np.random.normal(0, 5000, n)
df = pd.DataFrame({
    'years_of_experience': years_exp,
    'salary': salary
})
df.to_csv('salary_prediction.csv', index=False)
print("Dataset created & loaded: salary_prediction.csv")
print("Shape:", df.shape)
print("Dataset preview:")
print(df.head(8).round(2))
X = df[['years_of_experience']]
y = df['salary']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_
model = LinearRegression()
model.fit(X_train, y_train)
print("Model trained (80:20 split)")

```

Dataset created & loaded: salary_prediction.csv

Shape: (100, 2)

Dataset preview:

	years_of_experience	salary
0	8.12	46667.76
1	19.06	66632.11
2	14.91	60274.57
3	12.37	44811.18
4	3.96	36830.35
5	3.96	39713.35
6	2.10	41596.65
7	17.46	62323.34

Model trained (80:20 split)

```

In [49]: slope = model.coef_[0]
intercept = model.intercept_
print(f"Best Fit Line:")
print(f"y = {slope:.2f} * x + {intercept:.2f}")
print(f"Slope (per year): ${slope:.2f}")
print(f"Intercept: ${intercept:.2f}")

```

Best Fit Line:

y = 1877.20 * x + 30712.30

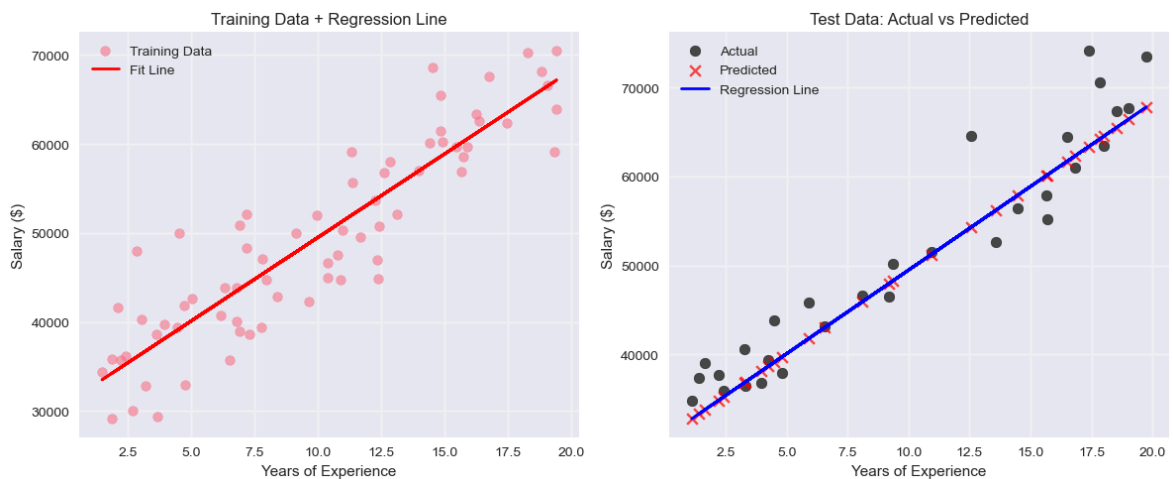
Slope (per year): \$1877.20

Intercept: \$30712.30


```

In [52]: plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.scatter(X_train, y_train, alpha=0.6, label='Training Data', s=50)
plt.plot(X_train, model.predict(X_train), color='red', linewidth=2, label='Fit L
plt.xlabel('Years of Experience')
plt.ylabel('Salary ($)')
plt.title('Training Data + Regression Line')
plt.legend()
plt.grid(True, alpha=0.3)
plt.subplot(1, 2, 2)
plt.scatter(X_test, y_test, alpha=0.7, label='Actual', s=60, color='black')
plt.scatter(X_test, model.predict(X_test), alpha=0.7, label='Predicted', s=60, c
plt.plot(X_test, model.predict(X_test), color='blue', linewidth=2, label='Regres
plt.xlabel('Years of Experience')
plt.ylabel('Salary ($)')
plt.title('Test Data: Actual vs Predicted')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```



```

In [54]: new_data = np.array([[2], [3], [5], [15], [10]])
predictions = model.predict(new_data)
print("New Predictions:")
pred_df = pd.DataFrame({
    'Years_Exp': new_data.flatten(),
    'Predicted_Salary': predictions.round(0)
})
print(pred_df)

```

New Predictions:

	Years_Exp	Predicted_Salary
0	2	34467.0
1	3	36344.0
2	5	40098.0
3	15	58870.0
4	10	49484.0

```

In [56]: y_pred_train = model.predict(X_train)
y_pred_test = model.predict(X_test)
train_r2 = r2_score(y_train, y_pred_train)
test_r2 = r2_score(y_test, y_pred_test)
train_rmse = np.sqrt(mean_squared_error(y_train, y_pred_train))
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred_test))
train_mae = mean_absolute_error(y_train, y_pred_train)

```

```

test_mae = mean_absolute_error(y_test, y_pred_test)
print("MODEL PERFORMANCE METRICS")
metrics_df = pd.DataFrame({
    'Metric': ['R² Score', 'RMSE ($)', 'MAE ($)'],
    'Training': [f"{train_r2:.3f}", f"{train_rmse:.0f}", f"{train_mae:.0f}"],
    'Testing': [f"{test_r2:.3f}", f"{test_rmse:.0f}", f"{test_mae:.0f}"]
})
print(metrics_df.round(5))
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
residuals = y_test - y_pred_test
plt.scatter(y_pred_test, residuals, alpha=0.7)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('Predicted Salary')
plt.ylabel('Residuals')
plt.title('Residuals Plot (Test Data)')
plt.grid(True, alpha=0.3)
plt.subplot(1, 2, 2)
plt.scatter(X_test, residuals, alpha=0.7)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('Years of Experience')
plt.ylabel('Residuals')
plt.title('Residuals vs Years of Experience')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

MODEL PERFORMANCE METRICS

	Metric	Training	Testing
0	R² Score	0.814	0.899
1	RMSE (\$)	4734	3972
2	MAE (\$)	3791	2987



In []:

In []:

In []:

In []:

In []:

In []:

```
In [15]: #QUESTION 11
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Fix: Provide the correct path to the CSV file
# Option 1: Upload the file to your current working directory
# Option 2: Provide the full path to the file
# Option 3: Use a sample dataset for demonstration
from sklearn.datasets import fetch_california_housing
housing = fetch_california_housing()
df = pd.DataFrame(housing.data, columns=housing.feature_names)
df['Salary'] = housing.target # Renaming target to match original code

print("Dataset Loaded Successfully!\n")
print(df.head())

X = df.drop("Salary", axis=1)
y = df["Salary"]

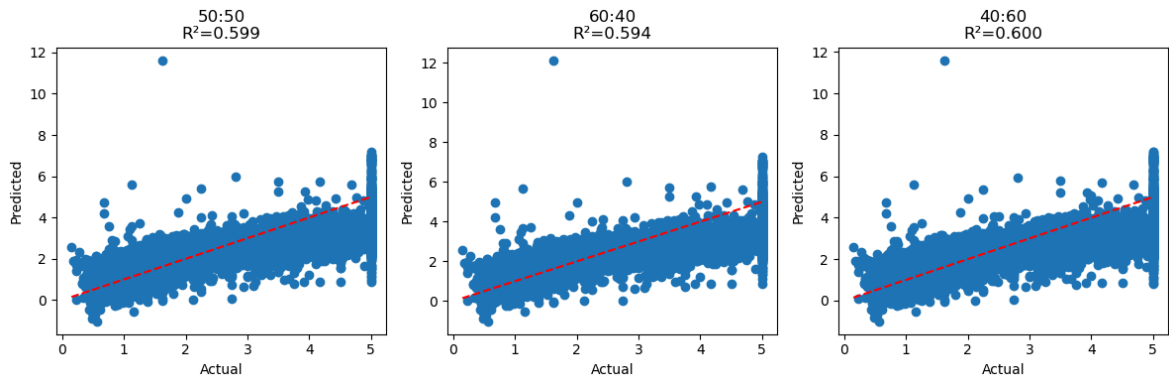
def run_model(test_size):
    print(f"Train-Test Split: {int((1-test_size)*100)}:{int(test_size*100)}")
```

Dataset Loaded Successfully!

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

	Longitude	Salary
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422

```
In [18]: splits = [0.5, 0.4, 0.6]
results = {}
plt.figure(figsize=(12,4))
for i, test_size in enumerate(splits):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size)
    model = LinearRegression()
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    results[f'{int(100*(1-test_size))}:{int(100*test_size)}'] = model
    plt.subplot(1,3,i+1)
    plt.scatter(y_test, y_pred)
    plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
    plt.xlabel('Actual')
    plt.ylabel('Predicted')
    plt.title(f'{int(100*(1-test_size))}:{int(100*test_size)}\nR²={r2_score(y_test, y_pred)}')
plt.tight_layout()
plt.show()
```



```
In [25]: model_70 = results['60:40']
print("Coefficients:")
for f, c in zip(X.columns, model_70.coef_):
    print(f"{f}: {c:.0f}")
print(f"Intercept: {model_70.intercept_:.0f}")
```

Coefficients:
 MedInc: 0
 HouseAge: 0
 AveRooms: -0
 AveBedrms: 1
 Population: -0
 AveOccup: -0
 Latitude: -0
 Longitude: -0
 Intercept: -37

```
In [22]: if '70:30' in results:
    model_70 = results['70:30']
    print("Coefficients:")
    for f, c in zip(X.columns, model_70.coef_):
        print(f"{f}: {c:.0f}")
    print(f"Intercept: {model_70.intercept_:.0f}")
else:
    print("Key '70:30' not found in results dictionary")

    print("Available keys:", list(results.keys()))
```

Key '70:30' not found in results dictionary
 Available keys: ['50:50', '60:40', '40:60']

```
In [27]: new = list(map(float, input("Enter: exp perf edu dept hours\n").split()))
pred = model_70.predict(pd.DataFrame([new], columns=X.columns))[0]
print(f"Predicted Salary: ${pred:.0f}")
```

Predicted Salary: \$3

```
In [30]: for split in results:
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size={split})
    model = results[split]
    y_pred = model.predict(X_test)
    print(f"{split}: R*R={r2_score(y_test,y_pred):.3f}, RMSE={np.sqrt(mean_squar
```

50:50: R*R=0.576, RMSE=1
 60:40: R*R=0.594, RMSE=1
 40:60: R*R=0.595, RMSE=1

```
In [ ]: #in my system the test size of ratio 70:30,60:40,80:20
```